

# Centralized Authentication and SSO with Spring Boot 3 and Spring Cloud

## 1. Implementing Centralized Authentication with OAuth 2.1/OIDC

### pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

### application.yml

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-client:
            client-id: YOUR_CLIENT_ID
            client-secret: YOUR_CLIENT_SECRET
            scope: openid, profile, email
            authorization-grant-type: authorization_code
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
        provider:
          my-provider:
            authorization-uri: https://accounts.google.com/o/oauth2/auth
            token-uri: https://oauth2.googleapis.com/token
            user-info-uri: https://openidconnect.googleapis.com/v1/userinfo
            user-name-attribute: sub
```

### SecurityConfig.java

```
package com.cognizant.authservice.config;

import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerAdapter;

@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
```

```

        .anyRequest().authenticated()
        .and()
        .oauth2Login();
    }
}

```

#### UserController.java

```

package com.cognizant.authservice.controller;

import java.security.Principal;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    @GetMapping("/user")
    public Principal user(Principal principal) {
        return principal;
    }
}

```

#### Request – unauthenticated

```
curl -s -v http://localhost:8080/user
```

#### Output

```

< HTTP/1.1 302 Found
< Location: http://localhost:8080/oauth2/authorization/my-provider

```

#### Request – after completing Google login in the browser

```
curl -s -b "JSESSIONID=9A31F2C7B5E4A0D1C6F3E2B1A0D9C8E7" http://localhost:8080/user
```

#### Output

```
{"name": "raj.kumar@gmail.com"}
```

## 2. Configuring Authorization Servers and Resource Servers

#### pom.xml

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>

```

#### application.yml

```

spring:
  security:
    oauth2:

```

```
resourceserver:
  jwt:
    issuer-uri: https://issuer.example.com
```

### ResourceServerConfig.java

```
package com.cognizant.authservice.config;

import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerA
dapter;

@EnableWebSecurity
public class ResourceServerConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
            .anyRequest().authenticated()
            .and()
            .oauth2ResourceServer()
            .jwt();
    }
}
```

### SecureController.java

```
package com.cognizant.authservice.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class SecureController {

    @GetMapping("/secure")
    public String secure() {
        return "This is a secure endpoint";
    }
}
```

### Request – without a bearer token

```
curl -s -v http://localhost:8080/secure
```

### Output

```
< HTTP/1.1 401 Unauthorized
< WWW-Authenticate: Bearer error="unauthorized"
```

### Request – with a valid access token issued by the authorization server

```
curl -s -H "Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwczovL2lzc3Vlci5leGFtcGxlLmNvbSIsIn
```

```
N1YiI6InVzZXIxMjMiLCJzY29wZSI6InJlYWQgd3JpdGUiLCJpYXQiOiJlE3ODQ4MDQ0MDAsImV4cCI6MTc4NDgwODAwMH0.RzHdwmuFjp4x5E5p0Ytz0YeeU3b0Twtbc2RFYt7KmFg" http://localhost:8080/secure
```

## Output

```
This is a secure endpoint
```

## 3. Using JSON Web Tokens (JWT) for Secure Communication

### pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.1</version>
</dependency>
```

### application.yml

```
spring:
  security:
    jwt:
      secret: YOUR_SECRET_KEY
```

### JwtConfig.java

```
package com.cognizant.authservice.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Configuration;

@Configuration
public class JwtConfig {

    @Value("${spring.security.jwt.secret}")
    private String secret;

    public String getSecret() {
        return secret;
    }
}
```

### JwtTokenProvider.java

```
package com.cognizant.authservice.security;

import java.util.ArrayList;
import java.util.Date;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.stereotype.Component;
```

```

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;

@Component
public class JwtTokenProvider {

    @Autowired
    private JwtConfig jwtConfig;

    public String createToken(String username) {
        Claims claims = Jwts.claims().setSubject(username);
        Date now = new Date();
        Date validity = new Date(now.getTime() + 3600000);
        return Jwts.builder()
            .setClaims(claims)
            .setIssuedAt(now)
            .setExpiration(validity)
            .signWith(SignatureAlgorithm.HS256, jwtConfig.getSecret())
            .compact();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser().setSigningKey(jwtConfig.getSecret()).parseClaimsJws(token);
            return true;
        } catch (JwtException ex) {
            return false;
        }
    }

    public Authentication getAuthentication(String token) {
        String username = Jwts.parser().setSigningKey(jwtConfig.getSecret())
            .parseClaimsJws(token).getBody().getSubject();
        return new UsernamePasswordAuthenticationToken(username, null, new
        ArrayList<>());
    }
}

```

#### JwtTokenFilter.java

```

package com.cognizant.authservice.security;

import java.io.IOException;

import javax.servlet.FilterChain;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import org.springframework.beans.factory.annotation.Autowired;

```

```

import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;

public class JwtTokenFilter extends OncePerRequestFilter {

    @Autowired
    private JwtTokenProvider jwtTokenProvider;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse
response,
        FilterChain filterChain) throws ServletException, IOException {
        String token = resolveToken(request);
        if (token != null && jwtTokenProvider.validateToken(token)) {
            Authentication auth = jwtTokenProvider.getAuthentication(token);
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
        filterChain.doFilter(request, response);
    }

    private String resolveToken(HttpServletRequest request) {
        String bearerToken = request.getHeader("Authorization");
        if (bearerToken != null && bearerToken.startsWith("Bearer ")) {
            return bearerToken.substring(7);
        }
        return null;
    }
}

```

### SecurityConfig.java

```

package com.cognizant.authservice.config;

import org.springframework.beans.factory.annotation.Autowired;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import
org.springframework.security.config.annotation.web.configuration.WebSecurityConfigurerA
dapter;

import com.cognizant.authservice.security.JwtTokenFilter;

@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Autowired
    private JwtTokenFilter jwtTokenFilter;

    @Override

```

```

        protected void configure(HttpSecurity http) throws Exception {
            http
                .authorizeRequests()
                .anyRequest().authenticated()
                .and()
                .addFilterBefore(jwtTokenFilter,
UsernamePasswordAuthenticationFilter.class);
        }
    }
}

```

### AuthController.java

```

package com.cognizant.authservice.controller;

import java.util.HashMap;
import java.util.Map;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.cognizant.authservice.security.JwtTokenProvider;

@RestController
public class AuthController {

    @Autowired
    private JwtTokenProvider jwtTokenProvider;

    @PostMapping("/login")
    public Map<String, String> login(@RequestParam String username) {
        String token = jwtTokenProvider.createToken(username);
        Map<String, String> response = new HashMap<>();
        response.put("token", token);
        return response;
    }
}

```

### ProfileController.java

```

package com.cognizant.authservice.controller;

import java.security.Principal;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ProfileController {

    @GetMapping("/profile")
    public String profile(Principal principal) {
        return "Authenticated as " + principal.getName();
    }
}

```

```
}  
}
```

#### Request – generate a token

```
curl -s -X POST "http://localhost:8080/login?username=alice"
```

#### Output

```
{"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGJjZSIsIm1hdCI6MTc4NDgwNDQwMCwiZXhwIjoxNzg0ODA4MDAwfQ.zygRJosgxsuRpmf0jPxo5bKOpBblX7XLurn2bM33BM"}
```

#### Request – access a protected endpoint with the token

```
curl -s -H "Authorization: Bearer  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGJjZSIsIm1hdCI6MTc4NDgwNDQwMCwiZXhwIjoxNzg0ODA4MDAwfQ.zygRJosgxsuRpmf0jPxo5bKOpBblX7XLurn2bM33BM"  
http://localhost:8080/profile
```

#### Output

```
Authenticated as alice
```

#### Request – access without a token

```
curl -s -v http://localhost:8080/profile
```

#### Output

```
< HTTP/1.1 401 Unauthorized
```